

ENERGETIKAI TECHNIKUM ÉS KOLLÉGIUM

Vizsgaremek

Suplex Gym

2026.

Szakma:
SZOFTVERFEJLESZTŐ ÉS -TESZTELŐ
5 0613 12 03

Készítették:
Nyirati Bálint
Tóth Levente

Tartalomjegyzék

I.	Bevezetés.....	4
II.	Témaválasztás indoklása	4
III.	Fejlesztői dokumentáció, technikai leírás	5
1.	Specifikáció.....	5
2.	Az alkalmazott fejlesztői eszközök	6
3.	Adatmodell.....	7
4.	Algoritmusok.....	14
5.	Tesztelési dokumentáció	17
6.	Szerepkör-alapú hozzáférés a weben	22
7.	IP-beállítások és hálózati konfiguráció	23
IV.	Felhasználói dokumentáció	23
8.	A program általános specifikációja, célja.....	23
9.	Rendszerkövetelmények.....	24
10.	A program telepítése	25
11.	A mobilalkalmazás használatának részletes leírása	28
12.	A webalkalmazás használatának részletes leírása	35
V.	API végpontok összefoglalása	42
VI.	Összegzés	48
VII.	Ábrajegyzék	49
VIII.	Irodalomjegyzék, hivatkozási jegyzék	50

Nyilatkozat

Alulírott Nyirati Bálint, Tóth Levente büntetőjogi felelősségem teljes tudatában nyilatkozom arról, hogy az itt szereplő vizsgaremek csoportmunka eredménye és sem részeiben sem egészében nem került még kereskedelmi forgalomba, ill. publikálásra, a GPL licenszelésű programrészek kivételével.

Paks, 2026. április 22.



Nyirati Bálint

Tóth Levente

I. Bevezetés

A mai világban különösen fontossá vált az elfogadás, illetve a másoknak való megfelelés és ebben meghatározó szerepet játszanak az emberek külső jegyei. A sportolás most éli eddigi legnagyobb fénykorát, és sokan azért kezdenek bele, hogy a megjelenésükön változtassanak. Kifejezetten populáris a testépítés, amely nem kizárólag a versenyzésről szól, hanem az egészséges életmód kialakításáról, az önfegyelem fejlesztéséről is.

Célunk egy könnyen kezelhető, univerzális edzőterem kezelő rendszer megvalósítása, melyben hatékony és átlátható módon kezeljük a beléptetést, a bérletek vásárlását és kezelését. A rendszer három összehangolt komponensre épül: egy webalkalmazásra, egy mobilalkalmazásra és egy API hívásokat fogadó kliensre. A felhasználók a mobilalkalmazásból vásárolhatnak jegyeket és bérleteket, a digitális QR-kóddal léphetnek be az edzőterembe, és nyomon követhetik aktív és lejárt bérleteik állapotát. A webalkalmazás egy bemutató felület biztosít a konditerembe, illetve adminisztratív célokat szolgál: az edzőterem munkatársai ezen keresztül kezelik a felhasználókat, a termékeket, a híreket és a berendezések állapotát.

A rendszer jelen állapotában egy saját, általunk kitalált edzőterem, a Suplex Gym kereteiben kerül bemutatásra. Megfelelő működés esetén a rendszer más edzőtermek számára is licenszelhetővé válna.

II. Témaválasztás indoklása

Vizsgaremekünk témáját a testépítés iránti érdeklődésünk, valamint a sporthoz való kötődésünk miatt választottuk. Több edzőterem is foglalkozik a digitális átállással és ennek lehetőségeivel, ám véleményünk szerint rendkívül primitív megoldásokat nyújtanak jelen állás szerint.

A témával kapcsolatos kutatásaink többsége az informatikai rendszerek felhasználói élményére és adatkezelésére összpontosít. Ezek szerint a felhasználók elsősorban megbízható, gyors és intuitív felületeket várnak el, a szolgáltatók pedig olyan rendszereket, amelyek minimalizálják a hibalehetőséget, átlátható kezelhetőséget adnak és automatizálják a repetitív folyamatokat.

Emellett vizsgáltuk az edzőtermek működésével kapcsolatos adminisztratív folyamatokat is. Számos helyen még mindig manuális vagy részben digitalizált rendszereket alkalmaznak a jegyek és bérletek kezelésére és a nyilvántartására. Ezek a módszerek gyakran időigényesek, és növelik az emberi hibák lehetőségét.

A megvizsgált piaci megoldások – mint például az egyszerűbb QR-kódos beléptetők – rendszerint nem kínálnak integrált mobilos vásárlási élményt. A mi rendszerünk ezt az igényt elégíti ki azzal, hogy a jegyvásárlástól a beléptetésig teljes körű digitális folyamatot biztosít.

III. Fejlesztői dokumentáció, technikai leírás

1. Specifikáció

A projekt fő feladata a felhasználók által megvásárolt jegyek pontos, és hibamentes kezelése, kiegészítve publikus weboldallal, és az azt tartalmazó adminisztrációs felülettel. A program során a következő funkciók érhetőek el:

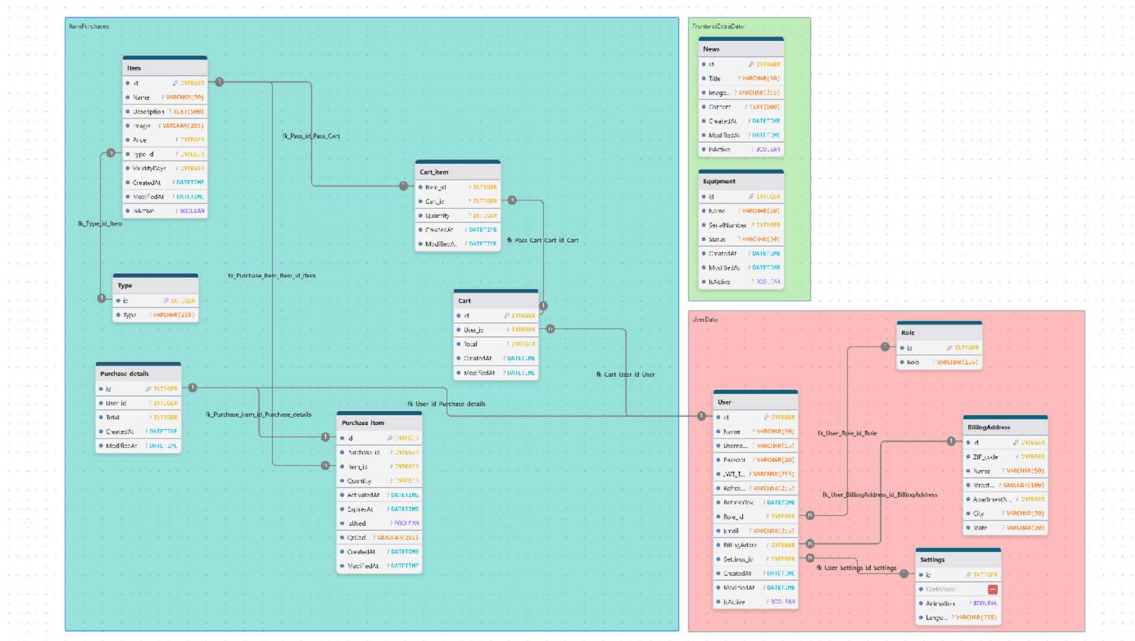
- A felhasználó regisztrálhat a mobilalkalmazásból. Regisztrációkor a rendszer visszaigazoló e-mailt küld.
- A felhasználó bejelentkezhet email-cím és jelszó kombinációval. A hitelesítés JWT access token és refresh token segítségével történik.
- A felhasználó megtekintheti és szerkesztheti profiladatait (név, felhasználónév, email-cím, jelszó, számlázási cím, alkalmazásbeállítások).
- A felhasználó böngészheti az elérhető jegyeket és bérleteket típus szerint szűrve (napi, havi, éves).
- A felhasználó kosárba helyezhet jegyeket, majd a vásárlást egyetlen gombnyomással véglegesítheti.
- A vásárolt jegyek azonnal aktívvá válnak, és QR-kód formájában jelennek meg a mobilalkalmazásban.
- A QR-kódok offline módban is elérhetők, mivel az adatokat az alkalmazás helyi tárhelyen is rögzíti.
- A lejárt jegyek megújíthatók közvetlenül az alkalmazásból.
- Az adminisztrátorok a webalkalmazáson keresztül kezelhetik a felhasználókat, jegyeket, híreket és a használt felszerelést.
- A publikus webhely megjeleníti a híreket, az árakat, a nyitvatartást és az általános tudnivalókat.

2. Az alkalmazott fejlesztői eszközök

A termékünk törekszik a bővíthetőségre és az egyszerű fejlesztői környezet megteremtésére, ehhez pedig a következő programokat és eljárásokat használtuk:

Kategória	Eszköz / Technológia
Programozási nyelv	TypeScript, C#
Web frontend	React 19, Vite 7, Tailwind CSS 4, shadcn/ui, react-router-dom, i18next, Turborepo
Mobil frontend	React Native, Expo SDK 54, NativeWind, Expo Router, Zustand, i18n-js
Backend	ASP.NET Core 8 Web API, Entity Framework Core 8
Adatbázis	SQLite (EF Core migrációkkal kezelve)
Hitelesítés	JWT Bearer token, BCrypt.Net-Next
E-mail	MailKit / MimeKit (SMTP, Gmail-kompatibilis)
QR-kód	QRCoder 1.4.3 (PNG generálás, Base64 tárolás)
IDE	Visual Studio 2022, Visual Studio Code
Verziókezelés	Git, GitHub
API-dokumentáció	Swagger / Swashbuckle

A feladatban alkalmazott adatbázis relációs diagrammja:



1. ábra - Adatbázis relációs diagrammja

Az adatbázis SQLite alapú, Entity Framework Core-ral kezelve. A séma három logikai csoportra bontható:

- A felhasználó adatait tároló UserData szekció
- A jegyek tárolására és vásárlására vonatkozó ItemPurchases
- Egyéb, a frontendhez tartozó kiegészítő adatok tárolására szolgáló FrontendExtraData

Az egyes táblák részletes leírása:

Users (Felhasználók)

A tábla a rendszerben regisztrált felhasználók adatait tárolja. Minden felhasználó egyedi azonosítóval rendelkezik, amely a többi táblával való kapcsolat alapjául szolgál.

Mező	Típus	Leírás
Id	int (PK)	Egyedi azonosító (autoincrement)
Name	string	Valódi név (max. 50 karakter)
Username	string	Felhasználónév (max. 15 karakter)
Email	string	E-mail cím (egyedi)
PasswordHash	string	BCrypt hash-elt jelszó (cost: 11)
JWT_token	string	Aktuálisan kiadott access token (max. 255 kar.)
RefreshToken	string	Megújítási token (7 nap érvényesség)
RefreshTokenExpiry	DateTime?	A refresh token lejárat ideje
Role_id	int (FK)	Hivatkozás a Roles táblára

Roles (Szerepkörök)

Három alapértelmezett szerepkör kerül feltöltésre az adatbázis inicializálásakor:

Id	Szerepkör	Hozzáférés
1	admin	Teljes hozzáférés minden admin végponthoz
2	user	Csak a saját adatokhoz és a vásárláshoz való hozzáférés
3	staff	Hírek és felszerelés kezelése; felhasználó- és termékkezelés nem elérhető

Items (Termékek/Jegyek)

A megvásárolható jegyeket és bérleteket tárolja. Minden termék rendelkezik típussal (napi / havi / éves), árengedménnyel, érvényességi napok számával és elérhetőségi állapottal.

Mező	Típus	Leírás
Id	int (PK)	Egyedi azonosító
Name	string	Termék neve (max. 30 karakter)
Description	string?	Rövid leírás (max. 500 karakter)
ImagePath	string?	Kép elérési útja vagy URL (max. 255 karakter)
Price	int	Ár forintban
Type_id	int (FK)	Hivatkozás a Types táblára
ValidityDays	int	Érvényesség napokban (pl. 1, 30, 90, 365)
IsActive	bool	Soft-delete jelző
CreatedAt	DateTime	Létrehozás időpontja
ModifiedAt	DateTime	Utolsó módosítás időpontja

Types (Jegytípusok)

Id	Típus neve	Érvényesség
1	napi	1 nap
2	havi	30 nap
3	éves	365 nap
4	szezonális	90 nap

Cart és CartItems (Kosár)

Minden felhasználóhoz legfeljebb egy aktív kosár tartozhat (egy-az-egyhez kapcsolat). A kosárban lévő tételek a CartItems táblában kerülnek tárolásra, jegy–kosár–mennyiség bontásban. A kosár megőrzi a félbehagyott vásárlásokat az alkalmazás újraindítása után is.

Mező (Cart)	Típus	Leírás
Id	int (PK)	Egyedi kosárazonosító
User_id	int (FK, unique)	Egy felhasználóhoz egy kosár
Total	int	Jelenlegi kosárösszeg forintban
CreatedAt / ModifiedAt	DateTime	Időbélyegek

Mező (CartItem)	Típus	Leírás
Id	int (PK)	Egyedi tételazonosító
Cart_id	int (FK)	Melyik kosárhoz tartozik
Item_id	int (FK)	Melyik jegyre vonatkozik
Quantity	int	Mennyiség
CreatedAt / ModifiedAt	DateTime	Időbélyegek

PurchaseDetails és PurchaseItems (Vásárlások)

A vásárlás során a kosár tartalma átkerül a PurchaseItems táblába. A PurchaseDetail egy vásárlási tranzakciót rögzít (felhasználó, végösszeg, időbélyeg), a PurchaseItems pedig az egyes jegyeket a következő adatokkal:

Mező (PurchaseItem)	Típus	Leírás
Id	int (PK)	Egyedi jegyazonosító
Purchase_id	int (FK)	Melyik vásárláshoz tartozik
Item_id	int (FK)	Az eredeti jegy/termék
Quantity	int	Darabszám
ActivatedAt	DateTime?	Aktiválás időpontja (vásárláskor állítódik be)
ExpiresAt	DateTime?	Lejárat időpontja (ActivatedAt + ValidityDays)

IsUsed	bool	Felhasználva jelző
QrCodeData	string?	Base64-kódolt PNG QR-kép
Created / Modified	DateTime	Időbélyegek

BillingAddress (Számlázási cím)

Mező	Típus	Leírás
Id	int (PK)	Egyedi azonosító
ZIP_code	int	Irányítószám
Name	string?	Számlázási név (max. 50)
StreetAddress	string?	Utca, házszám (max. 100)
ApartmentNumber	int	Emelet, ajtó
City	string?	Város (max. 30)
State	string?	Megye (max. 30)
IsActive	bool	Aktív-e

Settings (Felhasználói beállítások)

Mező	Típus	Leírás
Id	int (PK)	Egyedi azonosító
DarkMode	bool	Sötét mód be/ki
Animation	bool	Animációk be/ki
Language	string	Nyelv kódja (hu / en)
IsActive	bool	Aktív-e

News (Hírek)

Az edzőterem híreit tárolja. Csak adminisztrátorok hozhatnak létre és módosíthatnak híreket; a felhasználók kizárólag olvashatják azokat. Soft-delete alapú törlés: az IsActive mezőt false értékre állítva a hír nem jelenik meg a publikus listában.

Mező	Típus	Leírás
Id	int (PK)	Egyedi azonosító
Title	string	Cím (max. 50 karakter)
ImagePath	string	Borítókép URL (max. 255 karakter)
Content	string	Tartalom (max. 500 karakter)
IsActive	bool	Soft-delete jelző
CreatedAt / ModifiedAt	DateTime	Időbélyegek

Equipment (Berendezések)

Az edzőterem gépeit és berendezéseit tartja nyilván. Az adminisztrátorok az egyes eszközök állapotát három érték közül választva jelölhetik meg: operational (üzemel), maintenance (karbantartás alatt), out_of_order (nem üzemel). Soft-delete itt is alkalmazott.

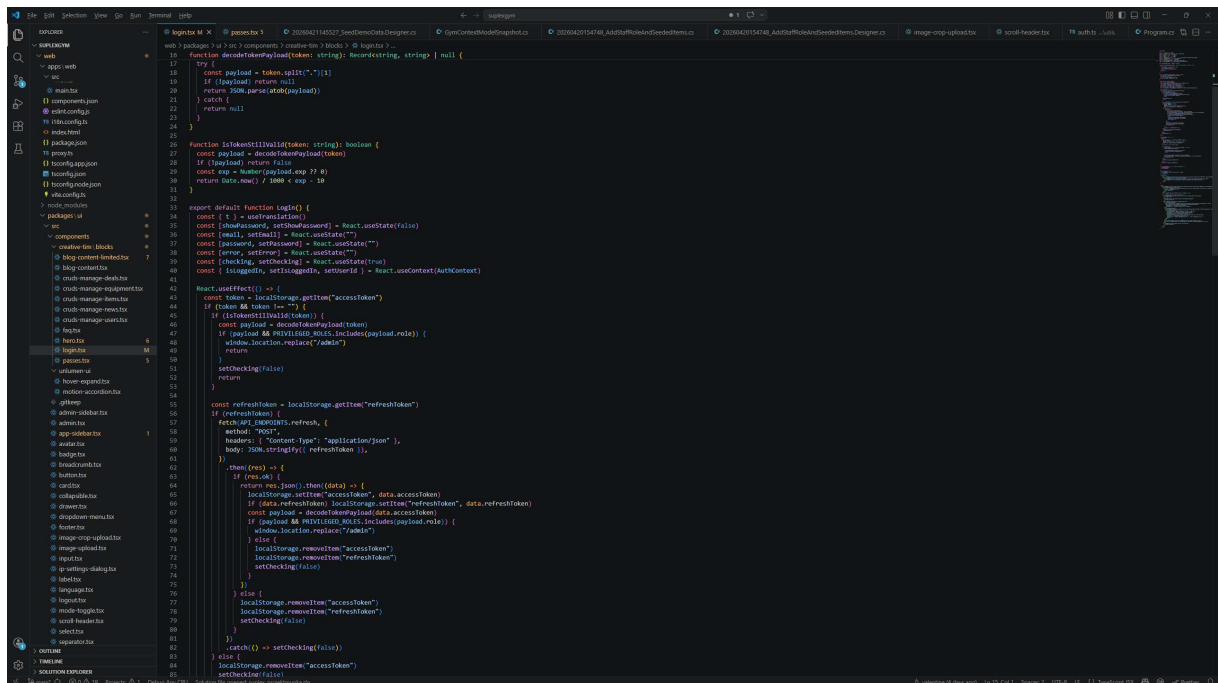
Mező	Típus	Leírás
Id	int (PK)	Egyedi azonosító
Name	string	Berendezés neve (max. 50)
SerialNumber	int	Sorozatszám
Status	string	operational maintenance out_of_order
IsActive	bool	Soft-delete jelző
CreatedAt / ModifiedAt	DateTime	Időbélyegek

4. Algoritmusok

Admin felületre való bejelentkezés:

Mivel a webes admin felületre kizárólag már létező admin vagy staff fiókok jelentkezhetnek be, az oldalon nem kínálunk regisztrációs lehetőséget. A bejelentkezési folyamathoz a felhasználó megadja az e-mail-címét és jelszavát a bejelentkezési űrlapon. A kliens POST kérést küld a /api/auth/login végpontra {email, password} törzsszel. A szerver megkeresi az e-mail alapján a felhasználót az adatbázisban. BCrypt.Verify() segítségével ellenőrzi a jelszót a tárolt hash ellen. Ha az ellenőrzés sikeres, új JWT access token és refresh token generál. A kliens a kapott accessToken és refreshToken értékeket localStorage-ba menti. A dekódolt JWT payload role mezőjét ellenőrizve, ha a felhasználó nem admin vagy staff, a rendszer elutasítja a belépést és hibát jelenít meg. Sikeres belépés esetén átirányítás a /admin útvonalra.

Megjegyzés: Minden egyes admin oldalra navigáláskor az alkalmazás automatikusan ellenőrzi, hogy a tárolt access token még érvényes-e (exp mező alapján). Ha nem, a refresh token segítségével megpróbálja megújítani. Ha az is lejárt, a tokeneket törli és a /login oldalra irányít.



```
function decodeToken(payload) {
  try {
    const payload = token.split('.')[1];
    if (!payload) return null;
    return JSON.parse(atob(payload));
  } catch (e) {
    return null;
  }
}

function isValidToken(token) {
  const payload = decodeToken(payload);
  if (!payload) return false;
  const exp = Number(payload.exp * 1000);
  return Date.now() / 1000 < exp;
}

export default function login() {
  const { t } = useTranslation();
  const [username, setUsername] = React.useState('');
  const [password, setPassword] = React.useState('');
  const [password2, setPassword2] = React.useState('');
  const [error, setError] = React.useState('');
  const [checking, setChecking] = React.useState(true);
  const [isLoading, setIsLoading] = React.useState(false);

  React.useEffect(() => {
    const token = localStorage.getItem('accessToken');
    if (token && token !== '') {
      if (isValidToken(token)) {
        const payload = decodeToken(payload);
        if (payload && payload.role === 'admin') {
          window.location.replace('/admin');
          return;
        }
      }
      setChecking(false);
      return;
    }

    const refreshToken = localStorage.getItem('refreshToken');
    if (refreshToken) {
      fetch(API_ENDPOINTS.refresh, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ refreshToken })
      })
        .then(res) => {
          if (res.ok) {
            return res.json().then(data => {
              localStorage.setItem('accessToken', data.accessToken);
              if (data.refreshToken) localStorage.setItem('refreshToken', data.refreshToken);
              const payload = decodeToken(payload);
              if (payload && payload.role === 'admin') {
                window.location.replace('/admin');
              } else {
                localStorage.removeItem('accessToken');
                localStorage.removeItem('refreshToken');
                setChecking(false);
              }
            });
          } else {
            localStorage.removeItem('accessToken');
            localStorage.removeItem('refreshToken');
            setChecking(false);
          }
        }
        .catch(() => setChecking(false));
    } else {
      localStorage.removeItem('accessToken');
      localStorage.removeItem('refreshToken');
      setChecking(false);
    }
  }, [refreshToken]);
}
```

2. ábra - Admin login ellenőrzés és access token megújítása

Jogosultság kezelés JWT token segítségével:

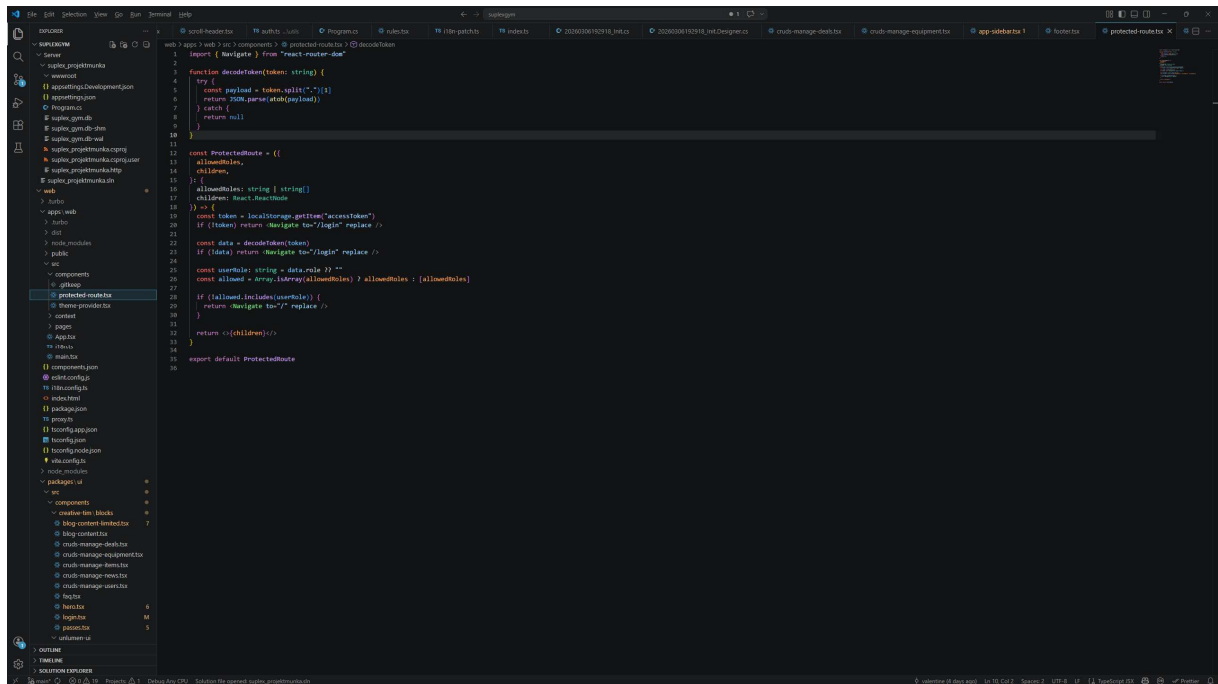
A jogosultságkezelés kétszintű: a szerveroldali ASP.NET attribútumokon és a kliensoldali token-dekódoláson alapul.

Szerveroldal:

Az AdminController osztály [Authorize(Roles = "admin,staff")] attribútummal van ellátva az osztály szintjén. Az admin-only végpontok (pl. felhasználókezelés, termékkezelés) külön [Authorize(Roles = "admin")] attribútumot kapnak metódus szinten. Ha a beérkező JWT token nem tartalmaz megfelelő szerepkört, a szerver 403 Forbidden választ küld.

Kliensoldal (web):

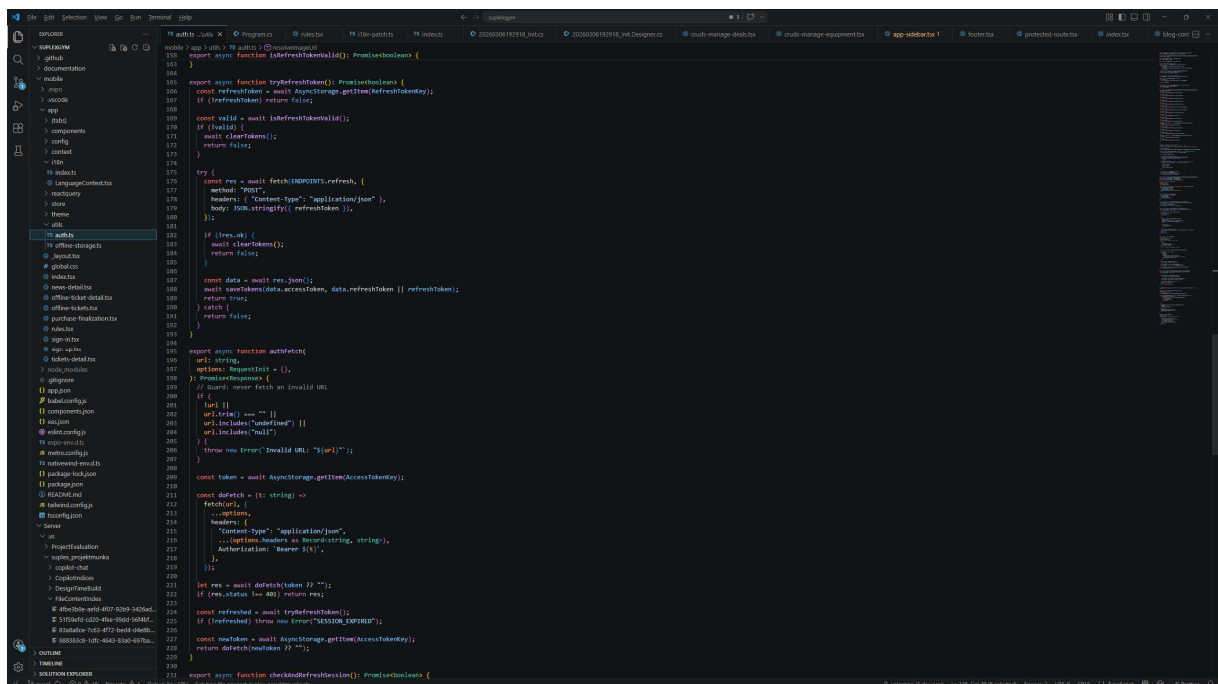
A kliens a localStorage-ban tárolt JWT tokenet az atob() segítségével base64-dekódolja és JSON formátumra hozza. A kinyert role értéket ellenőrzi, és csak admin/staff esetén engedi tovább a felhasználót az admin oldalra. A ProtectedRoute komponens elvégzi az átirányítást, ha a szerepkör nem megfelelő.



3. ábra - Admin login ellenőrzés és access token megújítása

Kliensoldal (mobil):

A mobilalkalmazás az AsyncStorage-ban tárolja a tokeneket. Az `authFetch()` segédfüggvény minden API kérés fejlécébe automatikusan beilleszti az `Authorization: Bearer {token}` sort. Ha a szerver 401-es választ küld, az `authFetch` automatikusan megkísérli a token megújítását a `tryRefreshToken()` függvény segítségével.



4. ábra - Admin login ellenőrzés és access token megújítása

Felszerelés állapotjelentés folyamata

Az adminok számára elérhető felület lehetővé teszi, hogy az edzőteremben dolgozó alkalmazottak valós időben jelentsék a kondigépek állapotát. Az Equipment fülön az admin listázni tudja az összes nyilvántartott felszerelést. A három lehetséges állapot:

operational – az eszköz működőképes

maintenance – az eszköz karbantartás alatt van

out_of_order – az eszköz meghibásodott, nem üzemel

Az állapotváltoztatás PUT kéréssel kerül elküldésre a `/api/admin/equipment/{id}/status` végpontra. A szerver validálja az állapot értékét (csak a három megengedett értéket fogadja el), majd frissíti a rekordot és rögzíti a ModifiedAt időbélyeget.

Hírek kezelése

A hírportál kezelése jogkörtől függően különböző funkciókat tesz elérhetővé. A felhasználók csak olvashatják a híreket a `/api/news` GET végponton keresztül, ahol az összes közzétett hír visszaadásra kerül csökkenő időrendi sorrendben.

Az adminok számára elérhető a `/api/news` végpont POST metódussal, amelyen keresztül új hírt hozhatnak létre. A kérés törzsében megadandó a cím és a tartalom. A szerver automatikusan rögzíti a létrehozás dátumát és a bejelentkezett admin azonosítóját. Hibás vagy hiányzó mező esetén a szerver 400-as Bad Request választ küld a hiányzó mezők listájával együtt. A webes admin felületen a hír szerkesztésekor lehetőség van képfeltöltésre is, amelyhez a rendszer a `POST /api/upload` végpontot hívja meg. A feltöltött fájl a `wwwroot/uploads/` mappába kerül, a visszaadott URL mentésre kerül a `News.ImagePath` mezőbe.

5. Tesztelési dokumentáció

A tesztelés során elsősorban fekete doboz (black-box) módszert alkalmaztunk, vagyis a tesztelés a rendszer külső viselkedésére összpontosított a belső implementáció ismerete nélkül. Az egyes tesztesetek tartalmazzák a tesztelt funkciót, a végrehajtott műveletet, a várt eredményt és a valós eredményt.

A tesztelést a következő rétegeken végeztük:

- Egységtesztek – az egyes API végpontok viselkedésének ellenőrzése Swagger UI segítségével (A teszttermékben meghagytuk a Swagger UI elérhetőségét tesztelés lehetőségére a következő címen: <http://localhost:5001/swagger/index.html>)
- Integrációs tesztek – a frontend-backend kommunikáció tesztelése valós kérésekkel
- Manuális funkcionális tesztek – a mobilalkalmazás és webalkalmazás felhasználói folyamatainak ellenőrzése
- Biztonsági tesztek – jogosulatlan hozzáférési kísérletek és hibás bemenetek tesztelése
- Határérték-tesztek – szélsőséges bemeneti értékek vizsgálata

1. Teszteset: Regisztráció hiányzó mezővel

Mező	Leírás
Funkció	Regisztrációs validáció a mobilalkalmazásban
Elvárás	A rendszer ne engedje a regisztrációt, ha kötelező mező hiányzik
Művelet	A regisztrációs űrlapon a felhasználónév mezőt üresen hagyjuk, a többi helyesen töltjük ki
Várt eredmény	Kliens oldali validáció, hibaüzenet megjelenítése, szerverhívás elmarad
Valós eredmény	Az ActivityIndicator nem indul el, "Töltsd ki az összes mezőt." üzenet jelenik meg
Eredmény	SIKERES

2. Teszteset: Bejelentkezés helytelen jelszóval

Mező	Leírás
Funkció	Hitelesítési hibakezelés
Elvárás	A szerver ne engedje be a helytelen jelszót megadó felhasználót
Művelet	Meglévő fiók e-mail-címét helyesen, jelszavát hibásan adjuk meg
Várt eredmény	401 Unauthorized válasz, hibaüzenet a felületen
Valós eredmény	A szerver 401-es státuszkóddal válaszol; "Hibás email vagy jelszó." felirat jelenik meg
Eredmény	SIKERES

3. Teszteset: Jegy megjelenítése internetkapcsolat nélkül

Mező	Leírás
Funkció	Offline elérhetőség
Elvárás	Az AsyncStorage-ban tárolt jegy offline módban is megjelenik
Művelet	Bejelentkezés internet-hozzáféréssel, majd internet kikapcsolása, majd jegy megnyitása
Várt eredmény	A jegy és a QR-kód renderelése hálózati hívás nélkül, Base64 adatból
Valós eredmény	A kép megjelenik, "Offline" badge látható, hálózati kérés nem indult
Eredmény	SIKERES

4. Teszteset: Admin végpont elérése nem megfelelő jogkörrel

Mező	Leírás
Funkció	Szerepkör-alapú hozzáférés-ellenőrzés
Elvárás	User jogkörű token nem férhet hozzá admin végpontokhoz
Művelet	User jogkörű JWT tokennel GET /api/admin/users kérés küldése Swagger UI-ban

Várt eredmény	403 Forbidden státuszkód
Valós eredmény	A szerver 403-as választ küld: {"message": "Forbidden. You do not have permission."}
Eredmény	SIKERES

5. Teszteset: Határérték-tesztek

Tesztelt adat	Bemeneti érték	Várt viselkedés	Valós eredmény
Felhasználónév hossza	16 karakter (limit: 15)	400 Bad Request (EF MaxLength validáció)	Szerver szintjén eldobja – SIKERES
Hír tartalma	501 karakter (limit: 500)	400 Bad Request	EF Core MaxLength szerver oldalon blokkol – SIKERES
Érvényesség (ValidityDays)	0 nap	Logikai hiba: azonnal lejárt jegy	Kezelni kell: minimum 1 napos validáció javasolt
Ár értéke	0 vagy negatív HUF	Logikailag érvénytelen	Az adatmodell int típusa nem tiltja; kliens oldali validáció javasolt
Rossz státuszérték felszerelés esetén	"broken" (nem engedélyezett)	400 Bad Request	A szerver ellenőrzi: csak operational/maintenance/out_of_order – SIKERES

Refresh token újrahasználata	Már használt refresh token	401 Unauthorized	A szerver új token-t ment el minden megújításkor, a régi érvénytelen – SIKERES
---------------------------------	-------------------------------	---------------------	--

6. Szerepkör-alapú hozzáférés a weben

A webalkalmazás háromszintű hozzáférés-ellenőrzést alkalmaz:

Szint	Megvalósítás	Leírás
Router védelem	ProtectedRoute komponens	Ellenőrzi a JWT token role mezőjét, 403 esetén /login- re irányít
UI szintű elrejtés	AdminSidebar userRole prop	A staff nem látja a user- és termékkezelési menüpontokat
Szerver szintű védelem	[Authorize(Roles=...)] attribútum	Minden admin végpont szerver oldalon is védi az adatot

Ez a háromszintű védelem biztosítja, hogy egyetlen réteg megkerülése esetén is a többi réteg védelmet nyújtson.

7. IP-beállítások és hálózati konfiguráció

A fejlesztői környezetben a webalkalmazás és a mobilalkalmazás is dinamikusan állítható API szerverrel kommunikál. Ez lehetővé teszi, hogy az alkalmazások bármely hálózaton futtassanak, ahol a backend elérhető.

Webalkalmazás (IpSettingsDialog)

A főoldal oldalsávjának alján egy fogaskerék ikon nyitja meg az IP-beállítások dialógust. A felhasználó megadhatja a backend szerver IP-címét (pl. 192.168.1.100). Az alkalmazás a localStorage-ban a web_server_ip kulcson tárolja az értéket. Az api-config.ts fájl getCachedApiBase() függvénye ezt az értéket olvassa, és <http://{ip}:5001> formátumú URL-t épít belőle. Ha nincs tárolt IP, az alkalmazás window.location.hostname értékét használja (ugyanazon gépen fut a backend és a frontend).

Mobilalkalmazás (DevIpSettings)

A főoldalon (csak fejlesztői buildben, __DEV__ feltétel mögött) egy fogaskerék gomb jelenik meg. Erre kattintva egy modal nyílik meg, ahol megadható a szerver IP-cím. Az IP az AsyncStorage-ban a dev_server_ip kulcson tárolódik. Az auth.ts getCachedApiBase() függvénye ezt az értéket olvassa.

IV. Felhasználói dokumentáció

8. A program általános specifikációja, célja

A Suplex Gym rendszer egy modern, digitális edzőterem-kezelő megoldás, amely két fő platformból áll: egy webalapú adminisztrációs felületből és egy mobilapplikációból. A rendszer célja az edzőtermi beléptetés, a jegy- és bérletkezelés, a felszerelések nyilvántartása, valamint a hírkommunikáció egyszerűsítése és digitalizálása.

Regisztráció és bejelentkezés: Az első használat alkalmával a felhasználónak regisztrálnia kell, amelyhez felhasználónevet, email-címet és jelszót kell megadnia. A regisztrációt email-megerősítés követi: a rendszer egy visszaigazoló levelet küld, amelyre kattintva aktiválható a fiók. Ezt követően a felhasználó bejelentkezhet, és hozzáférhet a rendszer funkcióihoz.

Jegy- és bérletvásárlás: A jegyek és bérletek vásárlása kizárólag a mobilapplikáción keresztül érhető el. A felhasználó négy különböző jegytípus közül választhat: napi belépőjegy, havi, szezonális (negyedéves), illetve éves bérlet. A megvásárolt jegyek offline módban is elérhetők, így az edzőterembe belépéshez nincs szükség aktív internetkapcsolatra.

QR-kód alapú beléptetés: Minden megvásárolt jegyhez egyedi QR-kód tartozik. A felhasználó a mobilappban a jegyre koppintva megjelenítheti a QR-kódot, amelyet az edzőterem személyzete leolvas a beléptetéshez. A QR-kód minden szükséges adatot tartalmaz a jegy érvényességének ellenőrzéséhez.

Hírportál: A rendszer tartalmaz egy hírportál-funkciót, amelyen keresztül az adminok (edzőtermi alkalmazottak) közleményeket, értesítéseket tehetnek közzé. A felhasználók a híreket olvashatják, de nem módosíthatják vagy törölhetik azokat.

Admin felület: Az adminisztrátori jogkörrel rendelkező felhasználók webes felületen keresztül kezelhetik a felszerelések állapotát, híreket hozhatnak létre, valamint hozzáférnek a rendszer adminisztrációs funkcióihoz. Az admin fiókot kizárólag a fejlesztők hozhatják létre biztonsági okokból.

9. Rendszerkövetelmények

Általános követelmények:

Operációs rendszer: Windows 10+, macOS 10.14+ vagy modern Linux disztribúció (pl. Ubuntu 20.04+)

Backend (C# / .NET):

.NET SDK 8.0 vagy újabb

Adatbázis: SQLite (automatikusan létrehozva az első indításkor)

Parancssoros eszközök (Terminál, PowerShell vagy CMD)

Frontend (React + Vite, webapplikáció):

Node.js 16.0 vagy újabb

Modern böngésző (Chrome, Firefox, Edge)

Internetkapcsolat: folyamatos kapcsolat szükséges

Mobilapplikáció (React Native):

Operációs rendszer: iOS 16 vagy újabb, Android 10 vagy újabb

Tárhely: legalább 100 MB szabad hely

Expo Go alkalmazás

Internetkapcsolat: szükséges a bejelentkezéshez és a vásárláshoz; a jegyek első bejelentkezés után offline módban is elérhetők

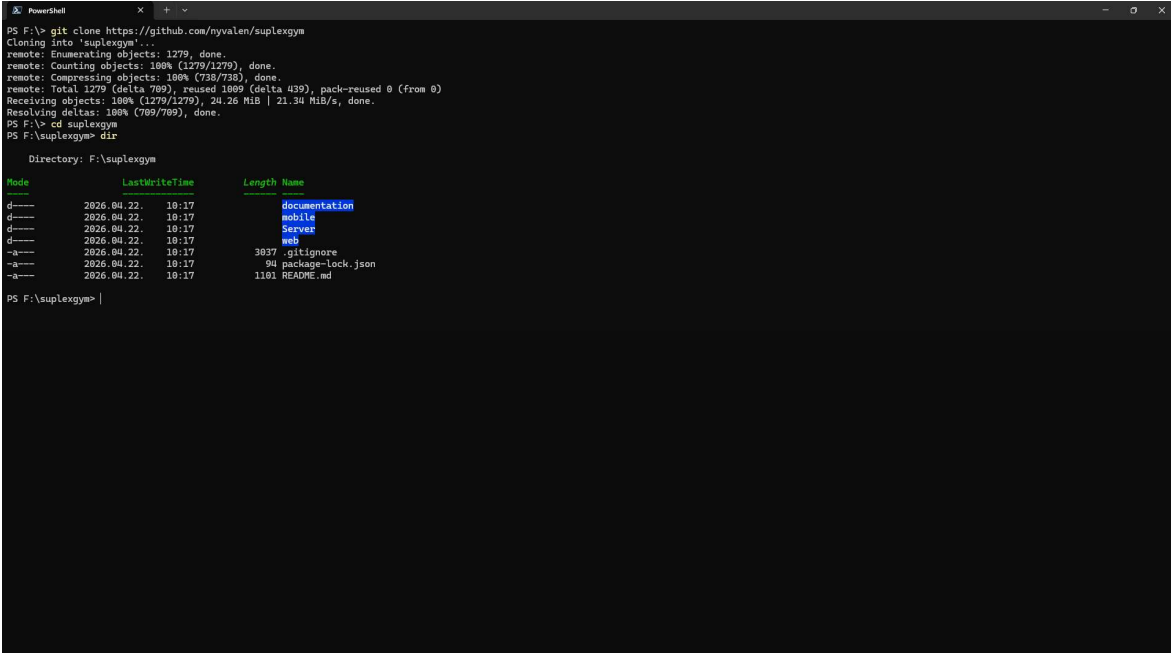
10.A program telepítése

A rendszer telepítéséhez szükség van a program GitHub repositorájára (<https://github.com/nyvalen/suplexgym>), amely tartalmazza a backend (szerver) és a frontend (webalkalmazás), valamint a mobilapplikáció forráskódját. Legalább 2 GB szabad tárhellyel kell rendelkezni a telepítőkönyvtárban.

1. lépés – Repository klónozása: Nyissunk egy PowerShell vagy CMD ablakot, és a kívánt helyen klónozzuk a repositoryt a saját rendszerükre az alábbi paranccsal:

git clone https://github.com/nyvalen/suplexgym

A kibontott mappán belül a következő struktúrát találjuk: Server (backend), mobile (mobilapplikáció) és web (webes frontend).



```
PS F:\> git clone https://github.com/nyvalen/suplexgym
Cloning into 'suplexgym'...
remote: Enumerating objects: 1279, done.
remote: Counting objects: 100% (1279/1279), done.
remote: Compressing objects: 100% (738/738), done.
remote: Total 1279 (delta 799), reused 1089 (delta 439), pack-reused 0 (from 0)
Receiving objects: 100% (1279/1279), 20.26 MiB | 21.34 MiB/s, done.
Resolving deltas: 100% (709/709), done.
PS F:\> cd suplexgym
PS F:\suplexgym> dir

Directory: F:\suplexgym

Mode                LastWriteTime         Length Name
----                -
d-----          2026.04.22.   10:17             documentation
d-----          2026.04.22.   10:17             mobile
d-----          2026.04.22.   10:17             server
d-----          2026.04.22.   10:17             web
-a-----          2026.04.22.   10:17           3037 .gitignore
-a-----          2026.04.22.   10:17           94 package-lock.json
-a-----          2026.04.22.   10:17          1181 README.md

PS F:\suplexgym> |
```

5. ábra - Repository klónozása

2. lépés – Backend és webapplikáció indítása: Navigáljunk a Server mappába:

cd suplexgym/Server/suplex_projektmunka

Ezek után adjuk ki az indítási parancsot:

dotnet run

A szerver a <https://localhost:5001> címen fut, illetve a gép ip címén is hostolva van, így egyszerre több számítógép/mobilkészülék is hozzáfér, ha ugyanazon hálózatra vannak csatlakozva; a zöld „Now listening” sor jelzi a sikeres indítást. A szerver indításával a webalkalmazás is elindul az előbb említett címeiken. Az induláskor automatikusan lefutnak az EF Core migrációk, amelyek létrehozzák az adatbázist és feltöltik az alapadatokkal (szerepkörök, jegytípusok, demo felhasználók, hírek, felszerelés).

Demo bejelentkezési adatok az indítás után:

Szerepkör	E-mail	Jelszó
Admin	admin@suplexgym.hu	Admin123!
Staff	peter@suplexgym.hu	Staff123!
User	anna@gmail.com	User123!

```
Windows PowerShell
PS C:\> cd suplexgym/Server\suplex_projektumunka
PS C:\suplexgym\Server\suplex_projektumunka dotnet run
Using launch settings from C:\suplexgym\Server\suplex_projektumunka\Properties\launchSettings.json...
Building
C:\suplexgym\Server\suplex_projektumunka\suplex_projektumunka.csproj : warning NU1902: Package 'MailKit' 4.3.0 has a known moderate severity vulnerability, https://github.com/advisories/GHSA-9J88-vyjs-vhgr
C:\suplexgym\Server\suplex_projektumunka\suplex_projektumunka.csproj : warning NU1902: Package 'MailKit' 4.3.0 has a known moderate severity vulnerability, https://github.com/advisories/GHSA-9J88-vyjs-vhgr
C:\suplexgym\Server\suplex_projektumunka\suplex_projektumunka.csproj : warning NU1902: Package 'MailKit' 4.3.0 has a known moderate severity vulnerability, https://github.com/advisories/GHSA-9J88-vyjs-vhgr
C:\suplexgym\Server\suplex_projektumunka\suplex_projektumunka.csproj : warning NU1902: Package 'MailKit' 4.3.0 has a known moderate severity vulnerability, https://github.com/advisories/GHSA-9J88-vyjs-vhgr
info: Microsoft.EntityFrameworkCore.Database.Command[20161]
      Executed DbCommand (0ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT COUNT(*) FROM "sqlite_master" WHERE "name" = '_EFMigrationsHistory' AND "type" = 'table';
info: Microsoft.EntityFrameworkCore.Database.Command[20161]
      Executed DbCommand (0ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT COUNT(*) FROM "sqlite_master" WHERE "name" = '_EFMigrationsHistory' AND "type" = 'table';
info: Microsoft.EntityFrameworkCore.Database.Command[20161]
      Executed DbCommand (0ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT "MigrationId", "ProductVersion"
      FROM "_EFMigrationsHistory"
      ORDER BY "MigrationId";
info: Microsoft.EntityFrameworkCore.Migrations[20005]
      No migrations were applied. The database is already up to date.
warn: Microsoft.AspNetCore.Server.Kestrel[0]
      warn: Overriding address(es) 'http://localhost:5001'. Binding to endpoints defined via IConfiguration and/or UseKestrel() instead.
info: Microsoft.Hosting.Lifetime[0]
      Now listening on: http://0.0.0.0:5001
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\suplexgym\Server\suplex_projektumunka
```

6. ábra - A backend szerver sikeres elindítása

3. lépés – Mobilapplikáció indítása: Fontos, hogy bármilyen mobilos megjelenítéshez a backend szervert futtató számítógépet és a mobilt megegyező hálózatra kell csatlakoztatni. Egy harmadik parancssori ablakban navigáljunk a mobilapp mappájába:

cd suplexgym/mobile

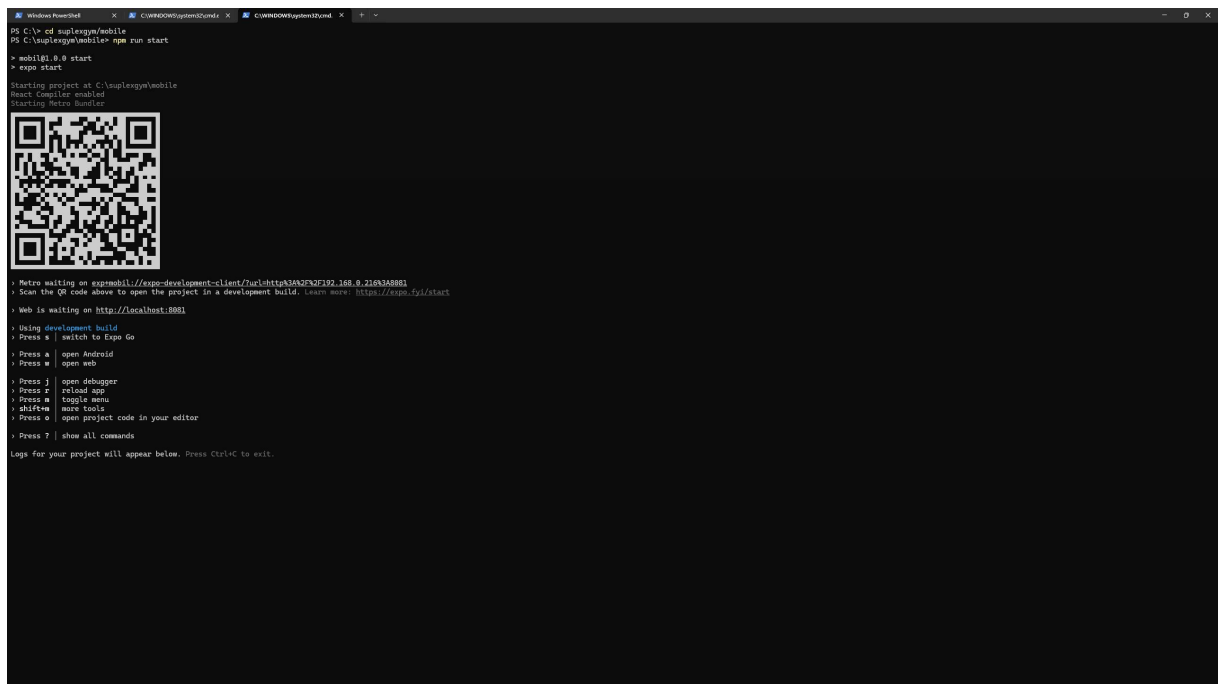
Majd telepítsük a függőségeket:

npm install

Ezek után adjuk ki az indító parancsot:

npm run start

A terminálban megjelenő QR-kódot Android rendszeren az Expo Go alkalmazással leolvasva, iOS eszközön a Kamera applikációval leolvasva indul el a program.

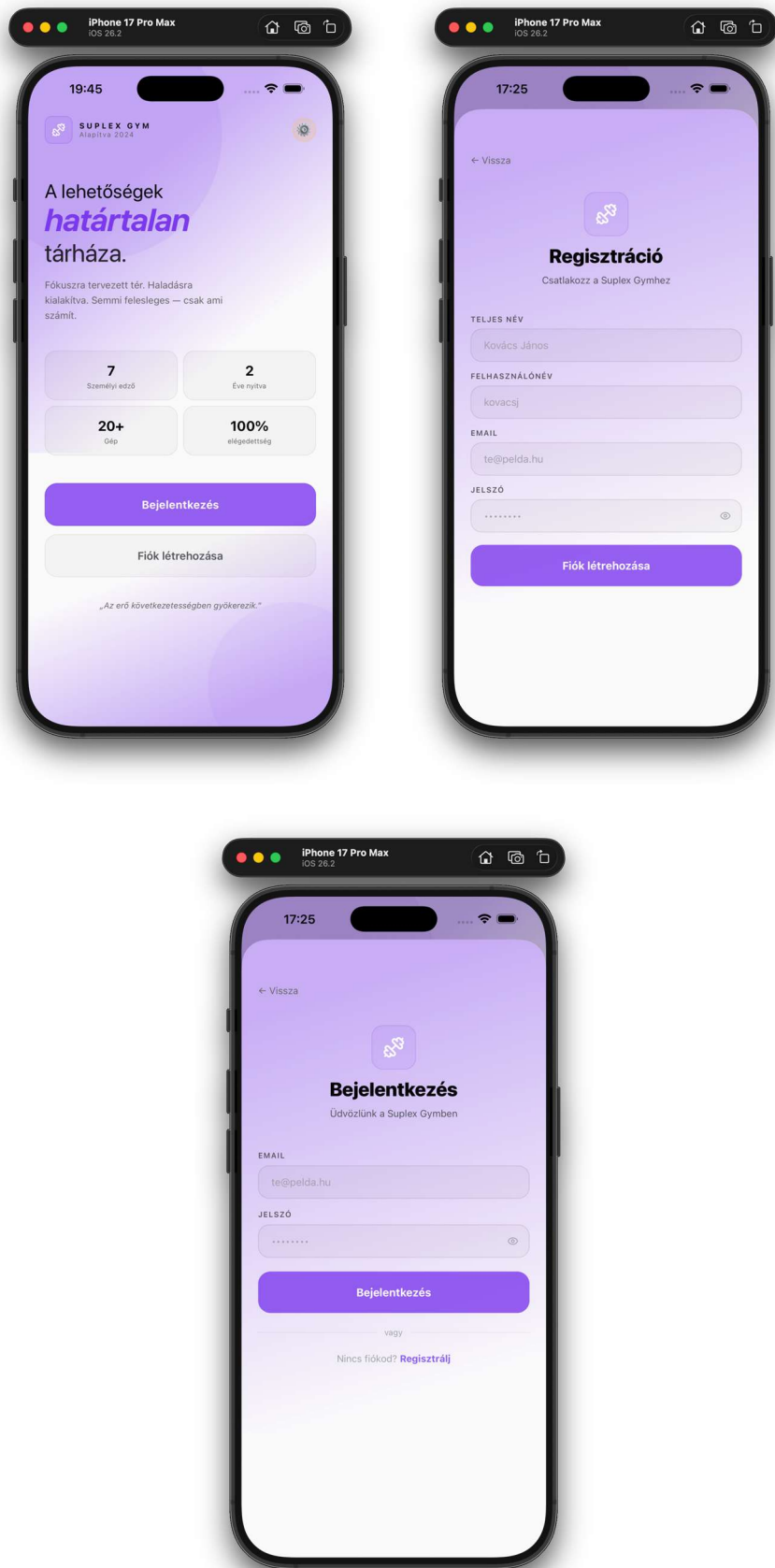


7. ábra - A mobilapplikáció indítása Expo Go segítségével (npm run start)

11.A mobilalkalmazás használatának részletes leírása

Regisztráció és bejelentkezés

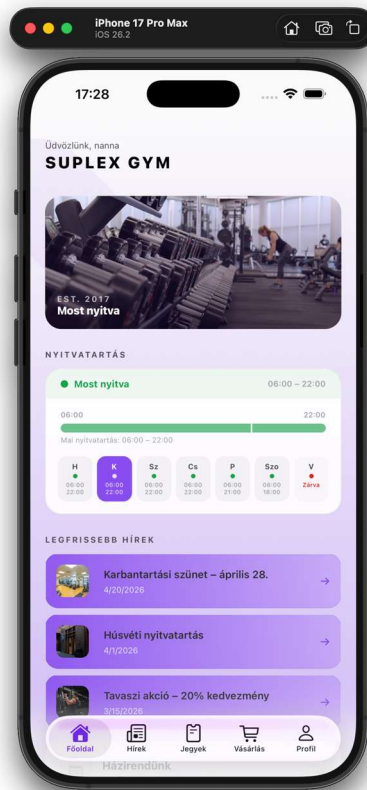
Az alkalmazás megnyitásakor egy üdvözlő képernyő fogadja a felhasználót. Új felhasználó esetén a „Fiók létrehozása” gombra kattintva megjelenik a regisztrációs űrlap, ahol meg kell adni a felhasználónevet, az email-címet és a jelszót. Sikeres regisztráció után a rendszer visszaigazoló emailt küld. Meglévő fiókkal rendelkező felhasználók közvetlenül bejelentkezhetnek email-címükkel és jelszavukkal.



8. ábra – Az üdvözlő, regisztrációs és bejelentkezési képernyő

Főoldal

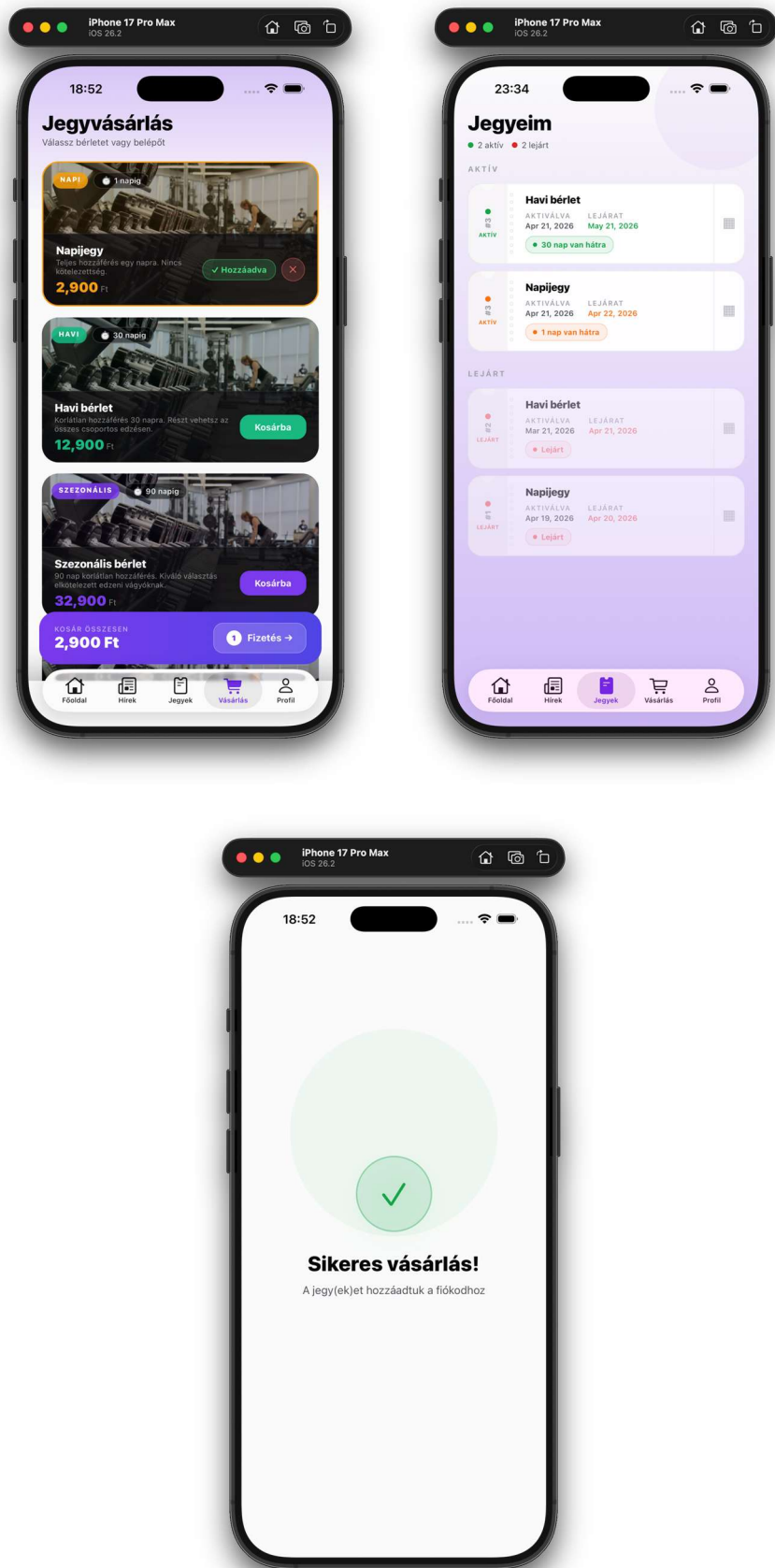
A Főoldal tartalmazza az edzőterem aktuális nyitvatartását, a mai nap nyitvatartási állapotát, a heti nyitvatartás összefoglalóját, és a legfrissebb három hírelemet. Az oldal alján megtekinthető az edzőterem szabályzata is.



9. ábra - Mobilalkalmazás főoldal

Jegyvásárlás

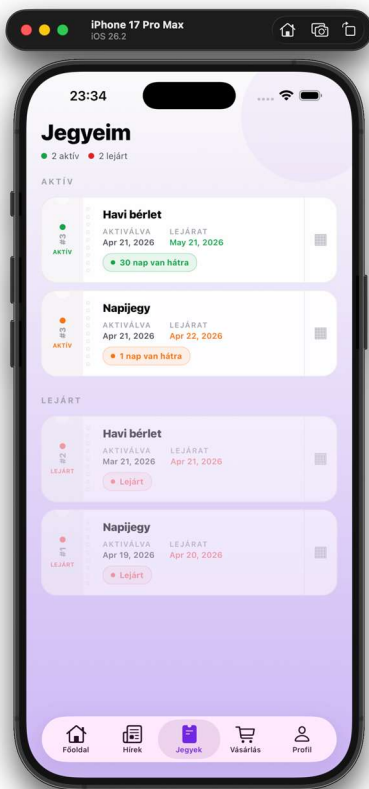
A Vásárlás fülön érhetők el az elérhető jegyek. A kosárban lévő jegyeket a kosár sáv jelzi az oldal alján. A Fizetés gomb megnyomásával a rendelés összesítő oldal jelenik meg. A Fizetés megerősítése gomb lenyomásával a vásárlás véglegesítésre kerül. Sikeres vásárlás után az a jegy azonnal aktiválódik és az app automatikusan a Jegyeim oldalra navigál.



10. ábra - Jegy vásárlás folyamata a mobilapplikációban

Jegyek és bérletek kezelése

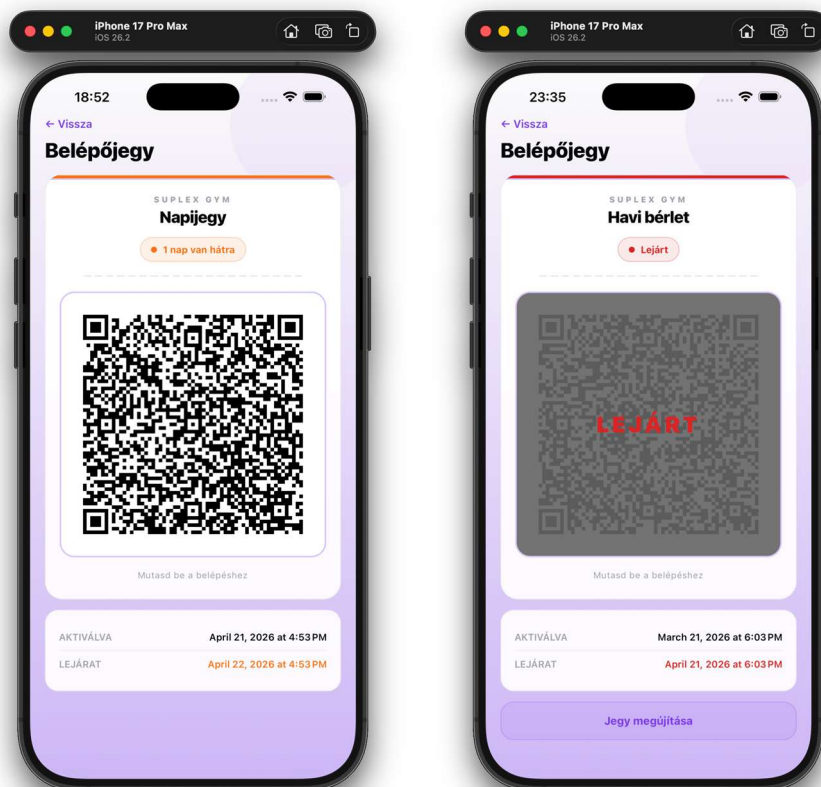
A Jegyek fülön az összes aktív és lejárt jegy látható, időrendben csoportosítva. Az aktív jegyek zöld jelzővel, a lejáró (7 napnál kevesebb) jegyek narancssárga és a lejárt jegyek piros jelzővel szerepelnek.



11. ábra - A jegyek és bérletek listázása a mobilapplikációban

QR-kód megjelenítése és beléptetés

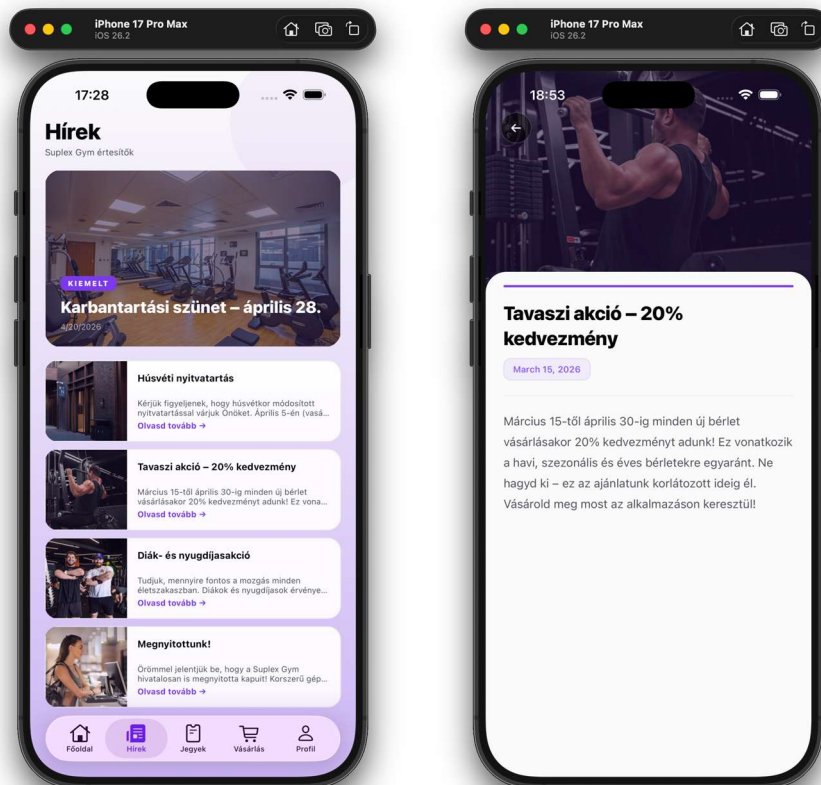
A jegyre koppintva megjelenik az adott jegyhez tartozó egyedi QR-kód. Ez a kód tartalmazza a jegy összes adatát: azonosítóját, típusát, lejárat dátumát és a tulajdonos adatait és hogy a Suplex Gym állította ki a jegyet. Az edzőterem beléptetőjénél a személyzet leolvassa a QR-kódot, a rendszer ellenőrzi az érvényességet, és sikeres validáció esetén engedélyezi a belépést. A QR-kód internetkapcsolat nélkül is megjeleníthető, mivel az adat az eszköz helyi tárhelyén is tárolódik. Lejárt jegy esetén a felhasználónak van lehetősége megújítani lejárt jegyét közvetlen ezen az oldalon.



12. ábra - Jegy QR-kódjának megjelenítése (offline módban is megegyezik) és lejárt jegy megújítása

Hírportál

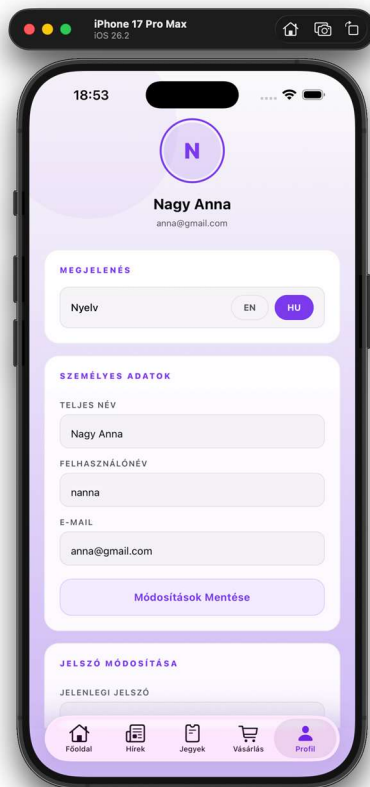
A „Hírek” fülön az edzőterem által közzétett közlemények érhetők el időrendi sorrendben, a legfrissebbtől visszafelé. Minden hírhez megjelenik a cím, a közzététel dátuma és a közzétevő admin neve. A felhasználók az egyes hírekre kattintva megtekinthetik azok teljes tartalmát, de módosítani vagy törölni nem tudják azokat. Új hírt kizárólag admin jogkörrel rendelkező felhasználók hozhatnak létre a webes adminisztrációs felületen keresztül.



13. ábra - A mobil hírportál felülete és kiválasztott hír megjelenítése

Profil

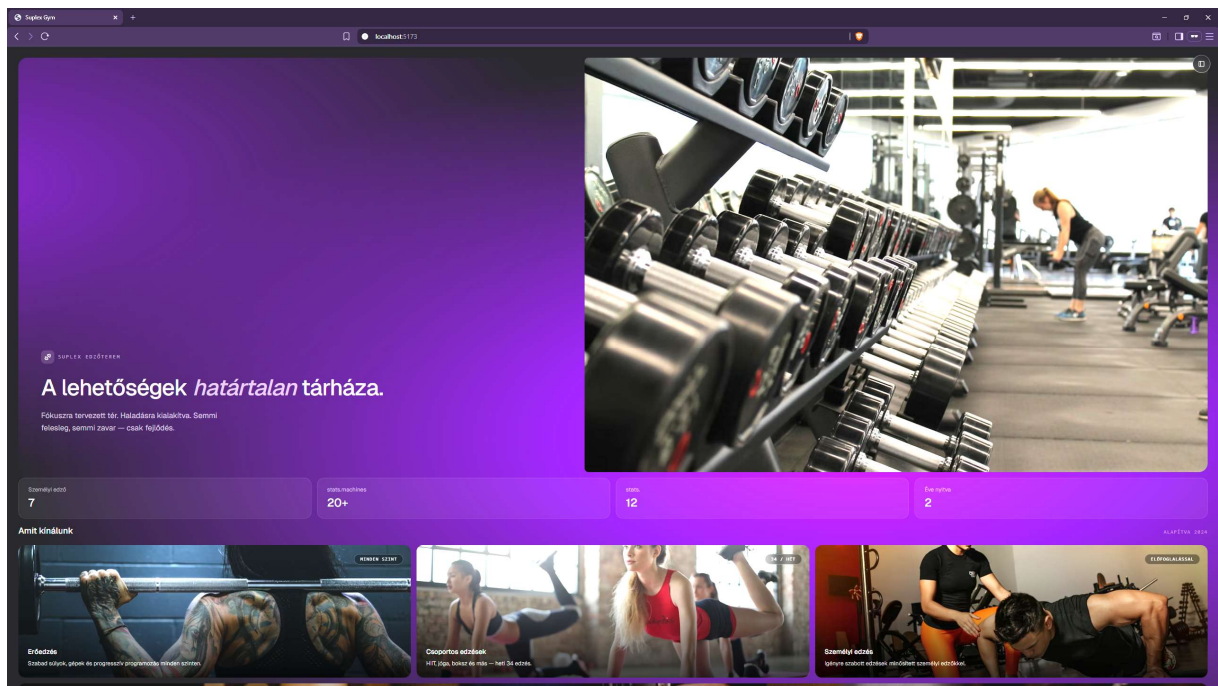
A Profil fülön a felhasználó képes módosítani a személyes adatait (név, felhasználónév, e-mail-cím), jelszót változtathat, megváltoztathatja a számlázási címét, angol vagy magyar nyelvre állíthatja alkalmazását, vagy pedig kijelentkezhet.



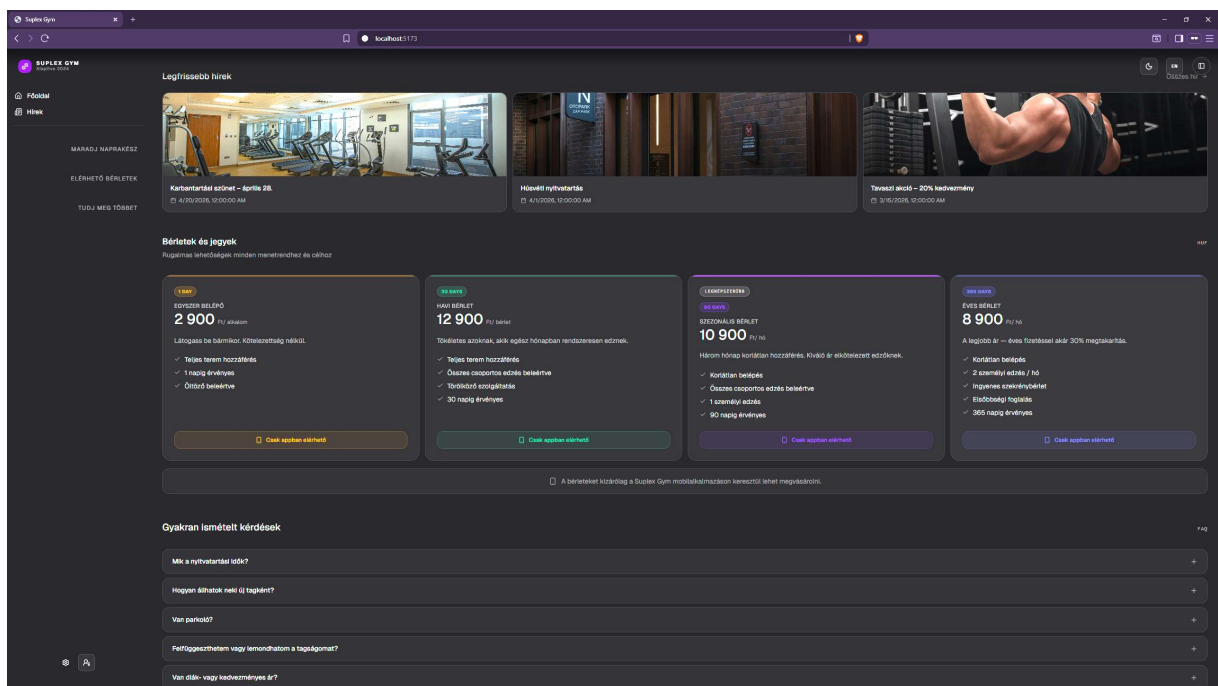
14. ábra - Profil felület

12.A webalkalmazás használatának részletes leírása

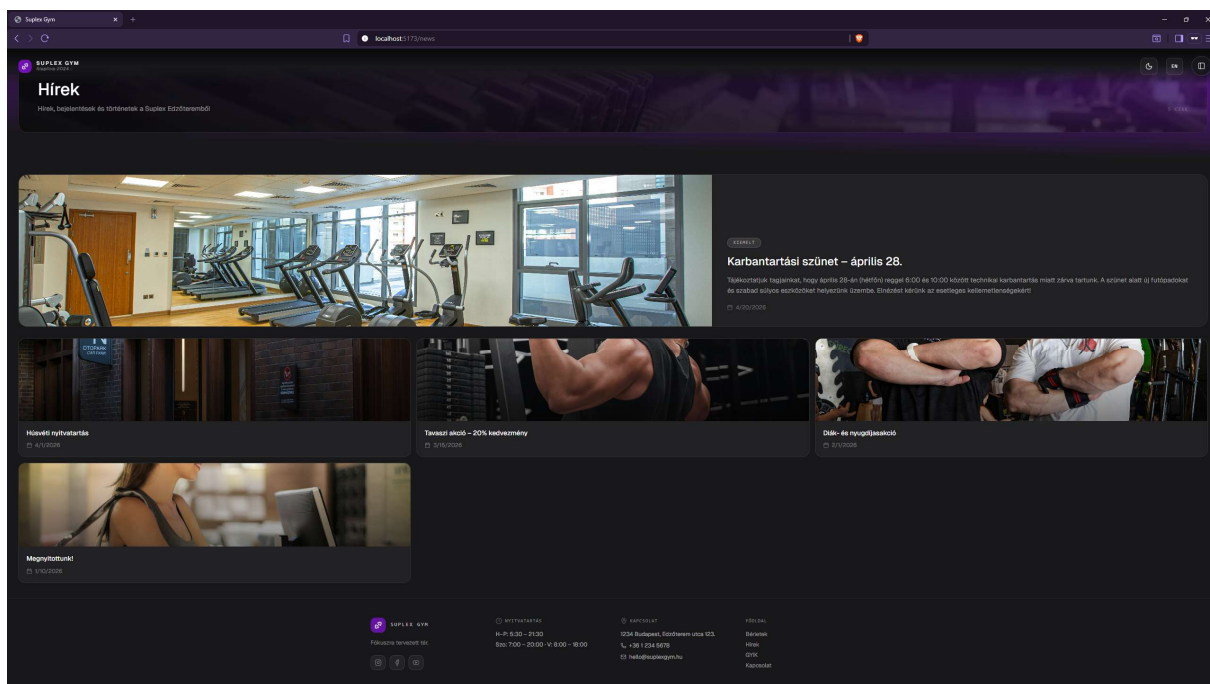
Adminisztrációs feladatok elvégzése mellett a weboldal tájékoztató célokat is szolgál. A weboldal megnyitásakor a főoldal fogad minket. Itt találhatóak a legfrissebb hírek, a különböző megvásárolható jegyek, bérletek és funkcióik, illetve a gyakran ismételt kérdésekre kaphat választ a felhasználó. A weboldal alján felvannak tüntetve elérhetőségek, illetve a nyitvatartás.



15. ábra - Webalkalmazás főoldal

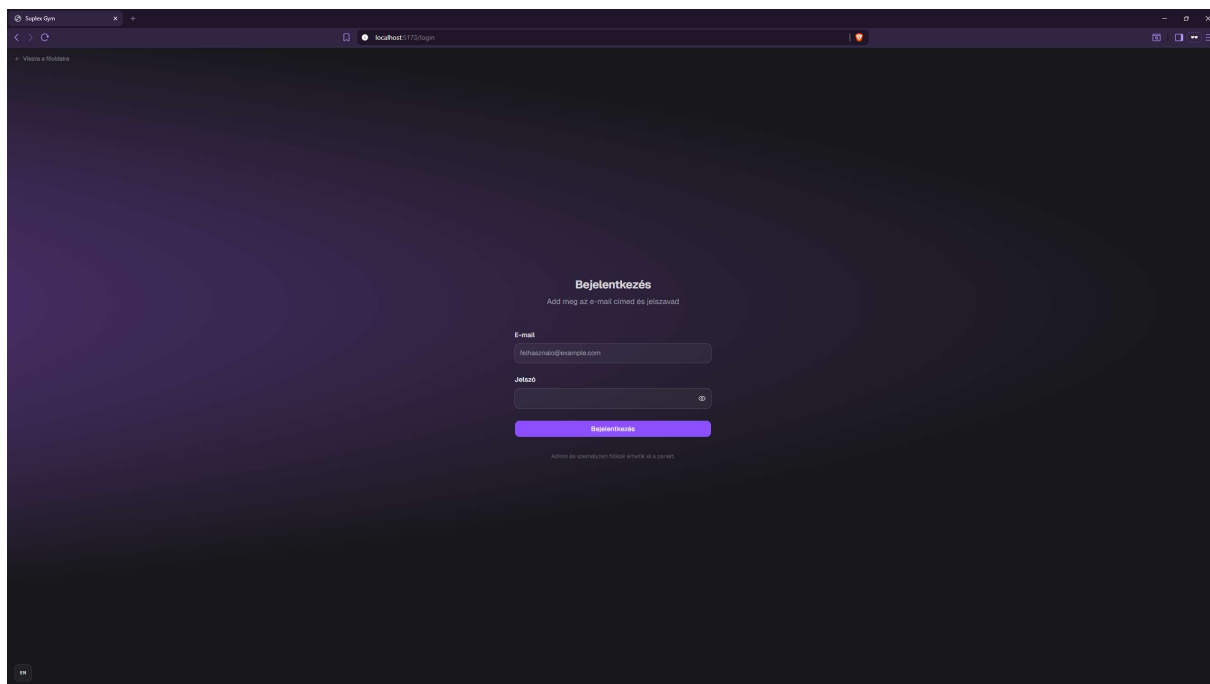


16. ábra - Friss hírek és bérletek szekció a weben



17. ábra - Összes hír listája a webalkalmazásban

A felhasználók számára elérhető a webes adminisztrációs felület, amit a jobb felső sarokban megnyitható gombbal nyitható oldalpanel alján rejtettünk el. A felületre való bejelentkezés mindenki számára elérhető, hiszen ha egy dolgozó a mobiljáról szeretne bejelenteni egy eszköz meghibásodását, annak a lehetőségét is biztosítanunk kell.

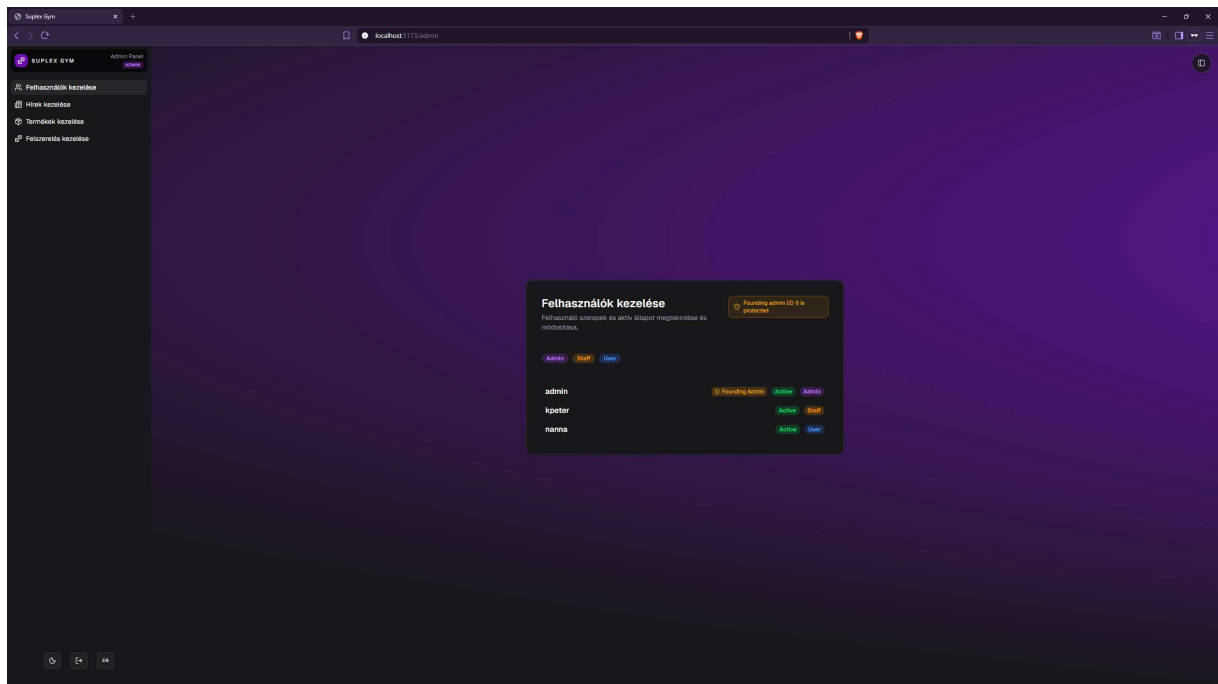


18. ábra - Admin bejelentkezési felület

Bejelentkezés után az oldalsávban található az összes kezelhető adat részekre bontva. Az ott dolgozó személyzet (staff) limitált hozzáféréssel rendelkezik; a hírekhez és a felszerelés fülhöz férnek hozzá. A változtatásokat a rendszer rögzíti a módosítás időpontjával együtt. Ez az átlátható állapotkövetés megkönnyíti a karbantartási feladatok koordinálását.

Felhasználók kezelése (CrudsManageUsers):

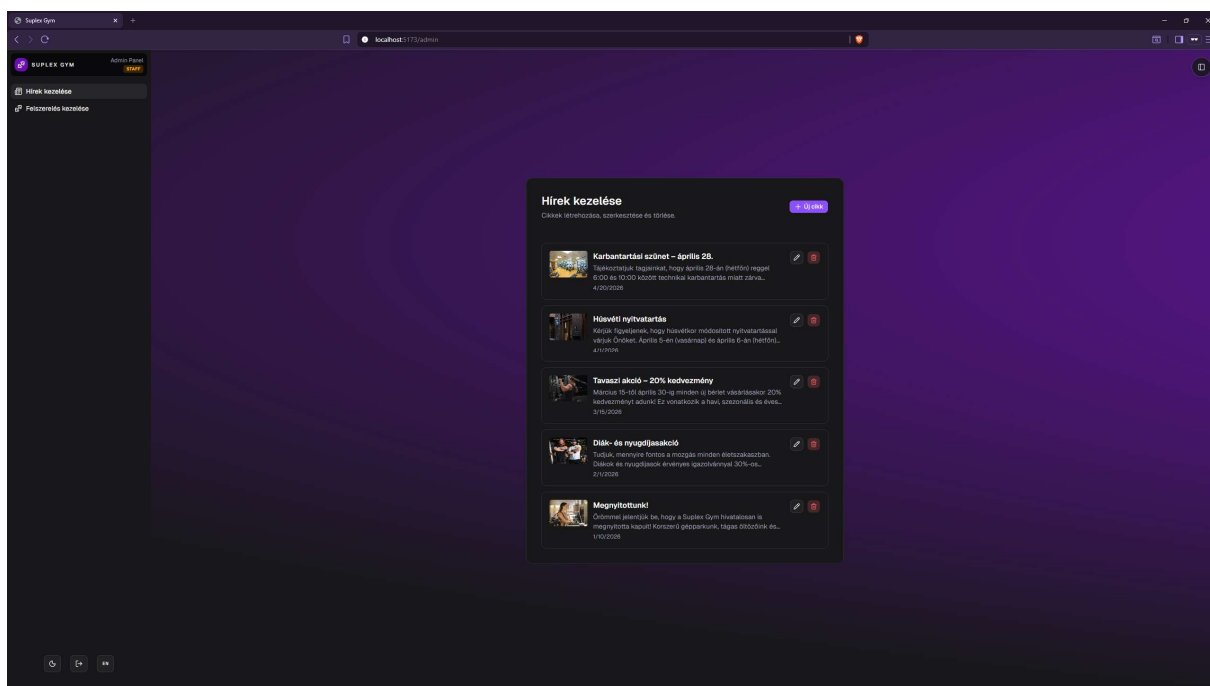
Csak admin jogkörrel érhető el. Az összes felhasználó listázható összezáráható kártyákon. Minden kártyán megjelenik a felhasználónév, szerepkör és aktív állapot. A kártyára kattintva kibővül és mutatja a teljes nevet, e-mailt, létrehozás dátumát. Szerepkör módosítása: legördülő listából választható (admin/staff/user), majd a Szerepkör mentése gombbal elküldhető. Aktiválás/deaktiválás: a Deaktiválás/Aktiválás gombbal az IsActive mező váltható. Founding admin védelem: az adatbázisban legkisebb azonosítójú admin felhasználó nem deaktiválható és szerepköre nem módosítható.



19. ábra - Az admin panel felhasználó kezelője

Hírek kezelése (CrudsManageNews):

Admin és staff jogkörrel is elérhető. Az összes aktív hír listázáshoz borítóképpel, címmel, dátummal és tartalomkivonattal. Új cikk létrehozása: cím és tartalom megadása kötelező, képfeltöltés opcionális. Feltöltés esetén képkivágás és forgatás is lehetséges a POST /api/upload végponton keresztül. Szerkesztés: az összes mező módosítható in-line, a változtatások PUT kéréssel menthetők. Törlés: soft-delete, a hírt DELETE kéréssel inaktíválja, de az adatbázisból nem törli.

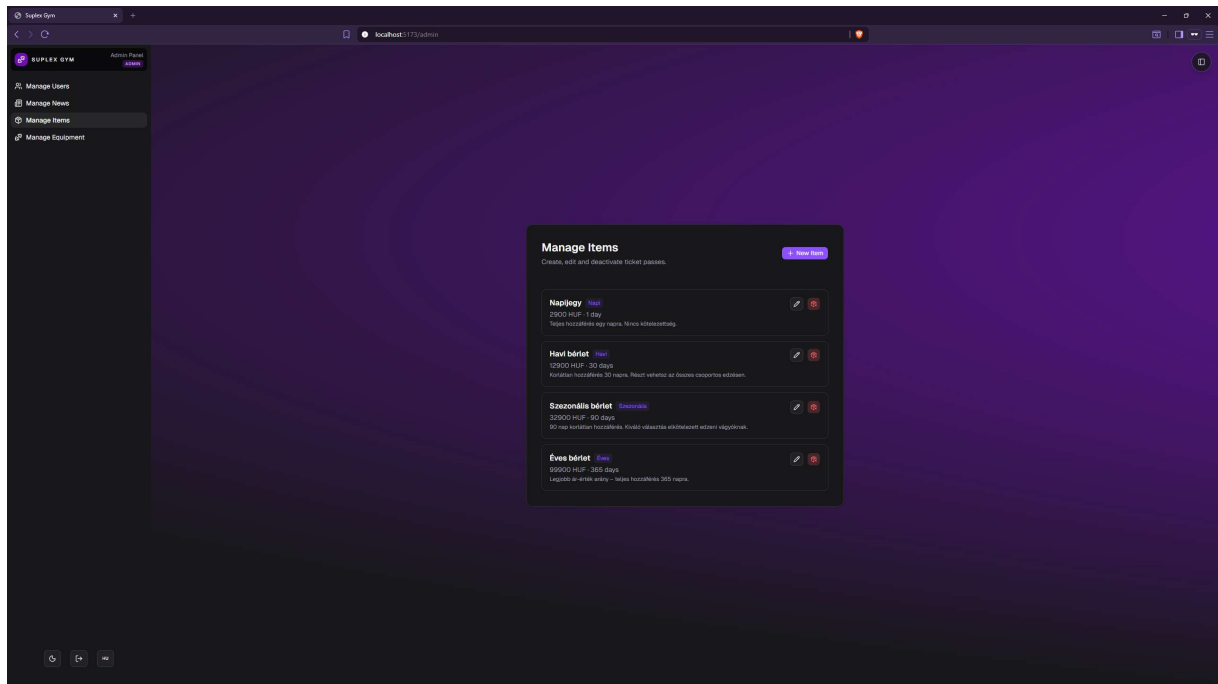


20. ábra - Az admin panel hírportál kezelője egy staff nézőpontjából

Termékek kezelése (CrudsManageItems):

Admin és staff jogkörrel is elérhető. Funkciók:

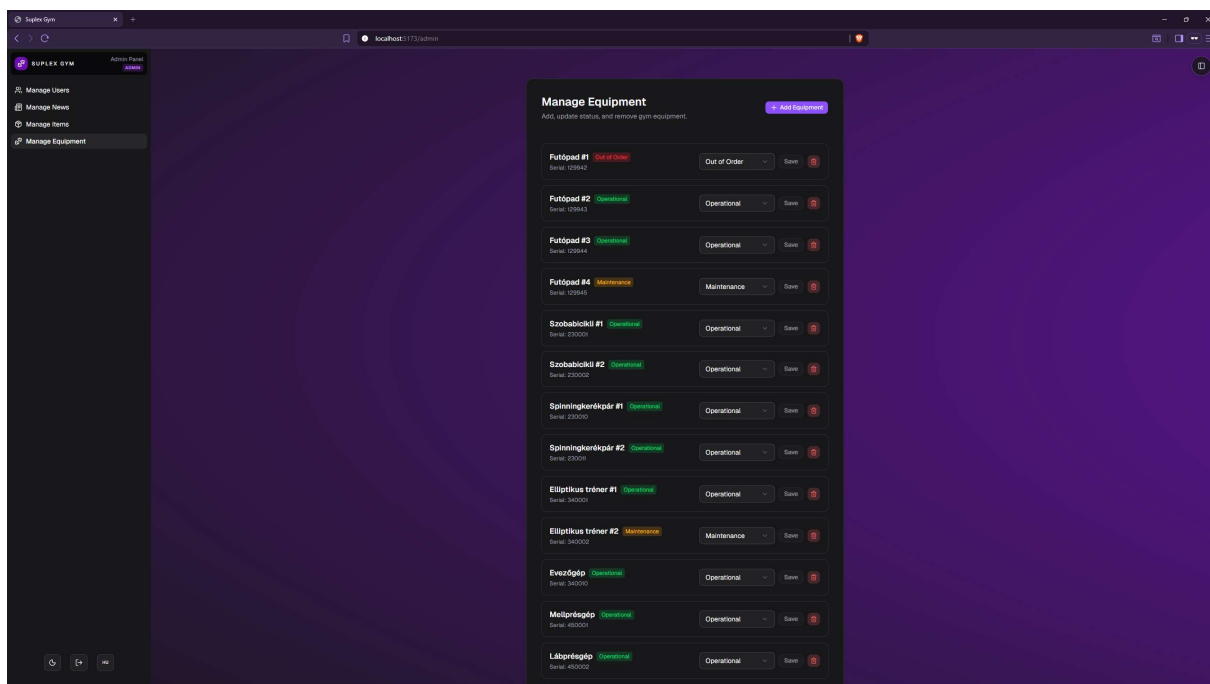
- Az összes aktív felszerelés listázása névvel, sorozatszámmal és színkódolt állapot-badge-dzsel.
- Állapot módosítása: legördülő listából kiválasztható az új állapot (Működő / Karbantartás alatt / Nem működik), majd a Mentés gombbal elküldhető PUT kéréssel.
- Csak a módosított állapotú eszközknél aktív a Mentés gomb (vizuálisan is jelezve).
- Új felszerelés hozzáadása: névvel, sorozatszámmal és kezdeti állapottal.



21. ábra - Az admin panel termék kezelője

Felszerelés kezelése (CrudsManageEquipment):

Admin és staff jogkörrel is elérhető. Az összes aktív felszerelést listázza névvel, sorozatszámmal. Állapot módosítása: legördülő listából kiválasztható az új állapot (Működő / Karbantartás alatt / Nem működik), majd a Mentés gombbal elküldhető PUT kéréssel. Csak a módosított állapotú eszköznél aktív a Mentés gomb (vizuálisan is jelezve). Új felszerelés hozzáadása: névvel, sorozatszámmal és kezdeti állapottal.



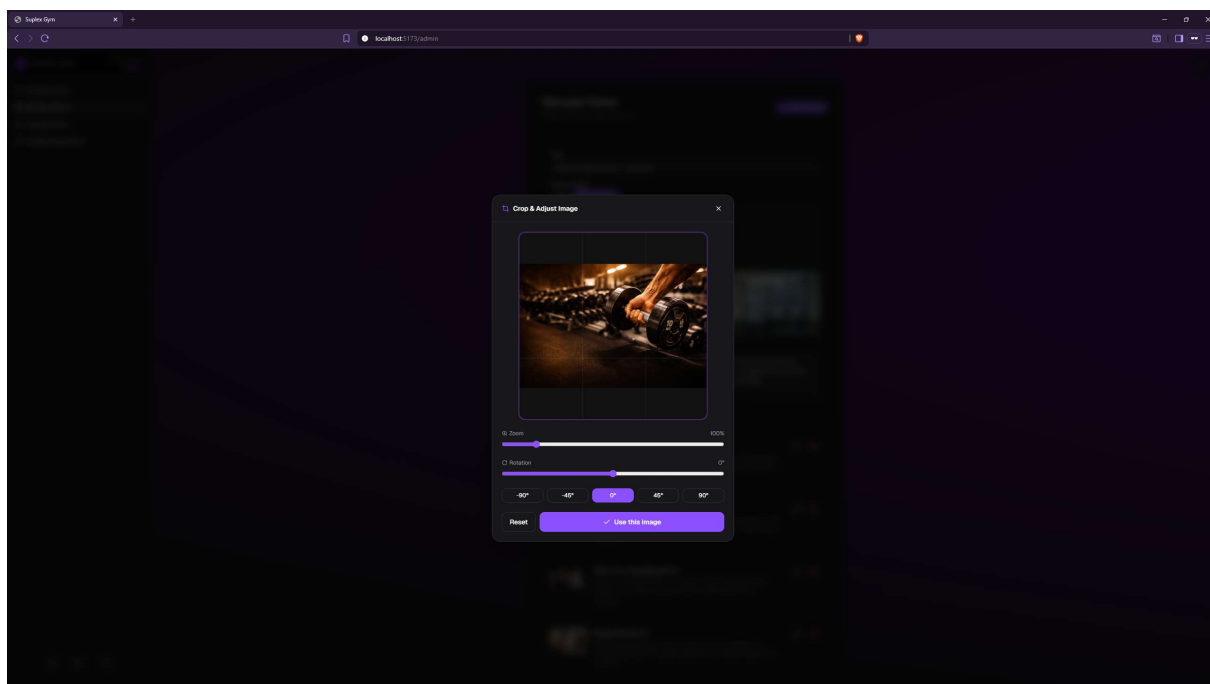
22. ábra - Az admin panel felszerelés kezelője

Képfeltöltés és kivágás (ImageCropUpload):

A hírek és termékek kezelésekor elérhető egy fejlett képkezelő komponens, amely két módban működik - URL mód: a felhasználó egy teljes URL-t adhat meg (pl. Unsplash linkre mutatva). Az URL azonnal előnézetbe kerül.

Feltöltés és kivágás mód: a felhasználó fájlt választ vagy húz be a dropzone területre. Ezt követően lehetősége van a kép mozgatására, nagyítására (50-400%) és forgatására (-180° - +180°). A kivágás után a kép JPEG formátumban kerül feltöltésre a szerverre.

A feltöltési folyamat: a kliens multipart/form-data POST kérést küld a /api/upload végpontra az Authorization headerrel. A szerver ellenőrzi a fájltypust és méretet (max. 10 MB), elmenti a wwwroot/uploads/ mappába, és visszaadja a /uploads/{uuid}.jpg relatív URL-t.



23. ábra - Kép feltöltése és módosítása

V. API végpontok összefoglalása

A teljes API a Swagger felületen tekinthető meg: <http://localhost:5001/swagger/index.html>

Hitelesítés (/api/auth)

Metódus	Végpont	Leírás	Jogkör
POST	/api/auth/register	Regisztráció (mobilból)	Nincs
POST	/api/auth/login	Bejelentkezés	Nincs
POST	/api/auth/refresh	Access token megújítása refresh tokennel	Nincs
POST	/api/auth/logout	Kijelentkezés (refresh token törlése)	Autentikált

Felhasználói profil (/api/user)

Metódus	Végpont	Leírás	Jogkör
GET	/api/user/profile	Saját profil lekérése	Autentikált
PUT	/api/user/profile	Profil frissítése (név, felhasználónév, e-mail)	Autentikált
PUT	/api/user/change-password	Jelszócsere	Autentikált
PUT	/api/user/billing-address	Számlázási cím mentése/frissítése	Autentikált
PUT	/api/user/settings	Beállítások mentése (nyelv, téma)	Autentikált

Jegyek és típusok (/api/items)

Metódus	Végpont	Leírás	Jogkör
GET	/api/items	Összes aktív jegy listázása (szűrhető ?typeId=N-nel)	Nincs
GET	/api/items/{id}	Egy jegy részletei	Nincs
GET	/api/items/types	Jegytípusok listája	Nincs

Kosár (/api/cart)

Metódus	Végpont	Leírás	Jogkör
GET	/api/cart	Saját kosár lekérése	Autentikált
POST	/api/cart/add	Jegy hozzáadása a kosárhoz	Autentikált
PUT	/api/cart/item/{id}	Kosárelem mennyiségének módosítása	Autentikált
DELETE	/api/cart/item/{id}	Kosárelem eltávolítása	Autentikált
DELETE	/api/cart/clear	Teljes kosár kiürítése	Autentikált

Rendelések (/api/orders)

Metódus	Végpont	Leírás	Jogkör
GET	/api/orders	Saját rendelési előzmények	Autentikált
GET	/api/orders/{id}	Egy rendelés részletei	Autentikált
POST	/api/orders/checkout	Vásárlás véglegesítése (kosár → jegyek)	Autentikált

POST	/api/orders/renew/{purchaseItemId}	Lejárt jegy megújítása	Autentikált
GET	/api/orders/ticket/{id}/qr	Jegy QR-kódjának lekérése	Autentikált

Hírek (/api/news)

Metódus	Végpont	Leírás	Jogkör
GET	/api/news	Összes aktív hír listázása (csökkenő dátum szerint)	Nincs
GET	/api/news/{id}	Egy hír részletei	Nincs
POST	/api/news	Új hír létrehozása	Admin + Staff
PUT	/api/news/{id}	Hír módosítása	Admin + Staff
DELETE	/api/news/{id}	Hír soft-delete törlése	Admin + Staff

Admin – Felhasználók (/api/admin)

Metódus	Végpont	Leírás	Jogkör
GET	/api/admin/users	Összes felhasználó listázása	Admin
GET	/api/admin/users/{id}	Egy felhasználó adatai	Admin
PUT	/api/admin/users/{id}	Szerepkör és aktív állapot módosítása	Admin
GET	/api/admin/roles	Szerepkörök listája	Admin
GET	/api/admin/users/{id}/tickets	Felhasználó jegyei	Admin
POST	/api/admin/users/{id}/tickets	Jegy manuális hozzáadása felhasználóhoz	Admin

Admin – Termékek és felszerelés

Metódus	Végpont	Leírás	Jogkör
POST	/api/admin/items	Új termék létrehozása	Admin
PUT	/api/admin/items/{id}	Termék módosítása	Admin

DELETE	/api/admin/items/{id}	Termék deaktiválása (soft-delete)	Admin
GET	/api/admin/equipment	Felszerelés listázása	Admin + Staff
POST	/api/admin/equipment	Új eszköz hozzáadása	Admin + Staff
PUT	/api/admin/equipment/{id}/status	Állapot módosítása	Admin + Staff
DELETE	/api/admin/equipment/{id}	Eszköz eltávolítása (soft-delete)	Admin + Staff

Képfeltöltés (/api/upload)

Metódus	Végpont	Leírás	Jogkör
POST	/api/upload	Kép feltöltése (multipart/form- data, max 10 MB)	Admin
DELETE	/api/upload?fileName=...	Feltöltött kép törlése	Admin

VI. Összegzés

A projekt során sikeresen megvalósítottuk a kitűzött célt: egy háromkomponensű edzőterem-kezelő rendszert hoztunk létre, amely az edzőterem tagjainak, az adminisztrátoroknak és a nyilvánosság számára is egységes, modern felületet biztosít.

A mobilalkalmazás a jegyvásárlástól a QR-kódos beléptetésig zökkenőmentes felhasználói élményt nyújt, és offline módban is képes megjeleníteni a korábban letöltött jegyeket. A webalkalmazás az adminisztrátori feladatokat teljes körűen lefedi, a rendszer biztonsági alapját pedig a JWT-alapú hitelesítés és a BCrypt jelszó-hash biztosítja.

A fejlesztés során a következő területeken szereztünk tapasztalatot, amelyeket a jövőben tovább kívánunk mélyíteni:

RESTful API tervezés és dokumentálás (Swagger)

Cross-platform mobilfejlesztés (React Native / Expo)

Monorepo-alapú webfejlesztés (Turborepo, shared UI könyvtár)

JWT-alapú autentikáció és refresh token kezelés

Offline-first tervezési minták (AsyncStorage, képkódolás Base64 formátumban)

Soft-delete adatkezelési stratégia alkalmazása

Olyan fejlesztési lehetőségek, amelyek a projekt keretébe nem fértek bele, de a jövőben megvalósítandók:

Valódi fizetési integráció (pl. Stripe vagy Barion API)

Push értesítések lejárató bérletek esetén

QR-kód szkennelés az admin webalkalmazásban, beléptetési naplóval, akár automatizált beléptető rendszer

Statisztikai irányítópult (látogatottság, bevétel, legjobb termékek)

E-mail alapú jelszó-visszaállítás

VII. Ábrajegyzék

1. ábra - Adatbázis relációs diagrammja	7
2. ábra - Admin login ellenőrzés és access token megújítása.....	15
3. ábra - Admin login ellenőrzés és access token megújítása.....	16
4. ábra - Admin login ellenőrzés és access token megújítása.....	16
5. ábra - Repository klónozása	25
6. ábra - A backend szerver sikeres elindítása.....	27
7. ábra - A mobilapplikáció indítása Expo Go segítségével (npm run start).....	28
8. ábra – Az üdvözlő, regisztrációs és bejelentkezési képernyő.....	29
9. ábra - Mobilalkalmazás főoldal	30
10. ábra - Jegy vásárlás folyamata a mobilapplikációban	31
11. ábra - A jegyek és bérletek listázása a mobilapplikációban	32
12. ábra - Jegy QR-kódjának megjelenítése (offline módban is megegyezik) és lejárt jegy megújítása.....	33
13. ábra - A mobil hírportál felülete és kiválasztott hír megjelenítése.....	34
14. ábra - Profil felület.....	35
15. ábra - Webalkalmazás főoldal	36
16. ábra - Friss hírek és bérletek szekció a weben.....	36
17. ábra - Összes hír listája a webalkalmazásban.....	37
18. ábra - Admin bejelentkezési felület	37
19. ábra - Az admin panel felhasználó kezelője	38
20. ábra - Az admin panel hírportál kezelője egy staff nézőpontjából	39
21. ábra - Az admin panel termék kezelője	40
22. ábra - Az admin panel felszerelés kezelője	41
23. ábra - Kép feltöltése és módosítása	42

VIII. Irodalomjegyzék, hivatkozásjegyzék

Microsoft – ASP.NET Core dokumentáció: <https://learn.microsoft.com/en-us/aspnet/core/>

Microsoft – Entity Framework Core: <https://learn.microsoft.com/en-us/ef/core/>

React – Dokumentáció: <https://react.dev/>

Expo – Dokumentáció: <https://docs.expo.dev/>

React Native – Dokumentáció: <https://reactnative.dev/docs/getting-started>

Tailwind CSS – Dokumentáció: <https://tailwindcss.com/docs>

shadcn/ui – Komponenskönyvtár: <https://ui.shadcn.com/>

creative-tim/ui – Komponenskönyvtár: <https://www.creative-tim.com/>

QRCoder – NuGet csomag: <https://github.com/codebude/QRCoder>

MailKit – NuGet csomag: <https://github.com/jstedfast/MailKit>

BCrypt.Net-Next – NuGet csomag: <https://github.com/BcryptNet/bcrypt.net>

Zustand – állapotkezelő: <https://zustand-demo.pmnd.rs/>

Expo Router – fájl-alapú navigáció: <https://expo.github.io/router/>